

AsliVastu — Complete Technical Reference

Interview preparation guide

Contents

AsliVastu — Complete Technical Reference (Interview Prep)	1
PART 1 — THE 60-SECOND PITCH	2
PART 2 — ARCHITECTURE (the big picture)	2
PART 3 — THE DATA PIPELINE (Python)	3
3.1 Stack	3
3.2 The scraper pattern	3
3.3 The one real scraper: <code>air_quality.py</code>	3
3.4 The merge step (<code>run_pipeline.py</code>)	3
PART 4 — THE SCORING ENGINE (<code>scoring.py</code>) — <i>the core IP</i>	4
4.1 The composite formula	4
4.2 Per-dimension formulas (know these cold)	4
4.3 Crime percentile (context, not just a raw number)	5
4.4 What I deliberately <i>didn't</i> build — the false-precision principle	5
4.5 Published methodology	5
PART 5 — PRICE CONTEXT (a non-composite dimension)	5
PART 6 — THE FRONTEND (Next.js)	6
6.1 Stack	6
6.2 Rendering strategy — SSR, and why it matters	6
6.3 API routes	6
6.4 The report page — key features	6
6.5 Performance & correctness details worth mentioning	7
PART 7 — THE “INDUSTRY” REDESIGN	7
PART 8 — DEPLOYMENT & OPS	7
PART 9 — THE MULTI-CITY EXPANSION (Delhi -> Bangalore)	8
PART 10 — LIKELY INTERVIEW QUESTIONS & STRONG ANSWERS	8
PART 11 — GLOSSARY (don't get caught out)	9
PART 12 — WHAT I'D DO NEXT (shows product thinking)	9
PART 13 — THE 3 THINGS TO LEAD WITH IN AN INTERVIEW	9

AsliVastu — Complete Technical Reference (Interview Prep)

Everything from “what is it” to the deepest implementation detail, plus the hard questions an interviewer will actually ask and how to answer them.

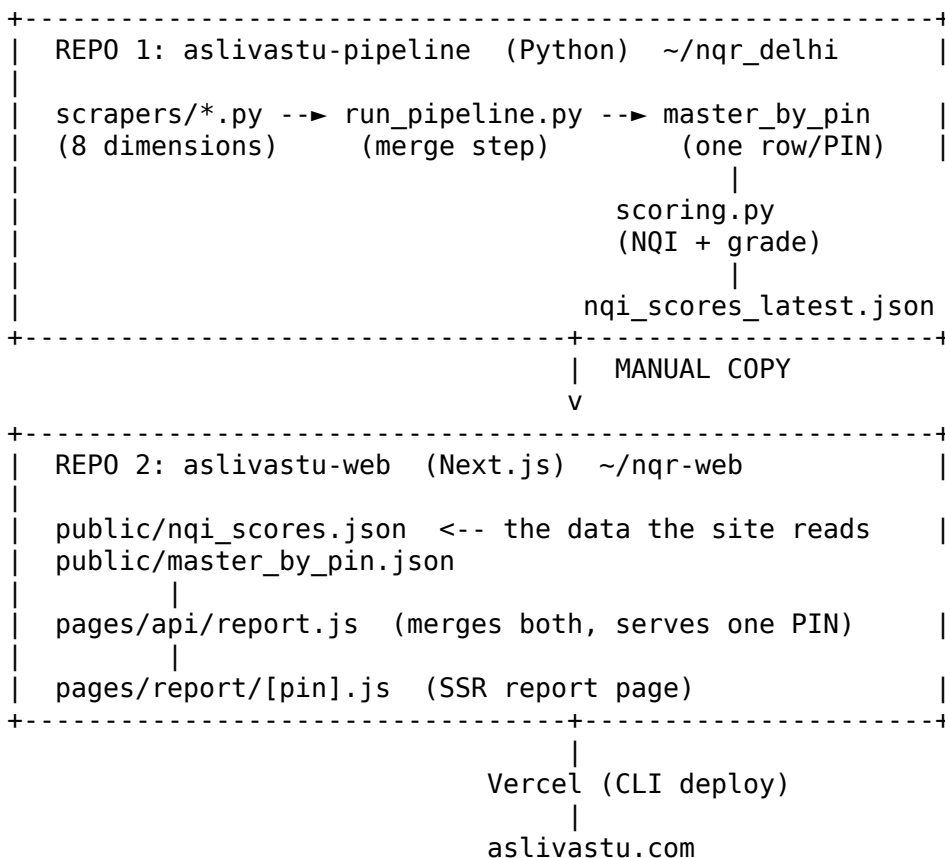
PART 1 — THE 60-SECOND PITCH

AsliVastu is a neighbourhood livability scoring product for Indian real estate. You type an area name or PIN code (e.g. “Whitefield” / 560066) and get a single 0-100 score called the **NQI (Neighbourhood Quality Index)**, a letter grade, and a full breakdown of the 8 dimensions behind it — safety, infrastructure, air quality, power, schools, water, roads and drainage — each sourced from government data.

The problem it solves: buying a home in India means checking a dozen disconnected government portals (police crime reports, CPCB air data, Jal Board supply records, municipal road surveys, stamp-duty circle rates) to answer a simple question: *is this area actually good to live in?* Brokers are incentivised to say yes. AsliVastu aggregates the public data into one honest, explainable number.

- **Live at:** aslivastu.com
- **Coverage:** 152 areas across 2 cities — Delhi NCR (86) and Bangalore (66)
- **Architecture:** two repos — a Python data pipeline + a Next.js frontend

PART 2 – ARCHITECTURE (the big picture)



Why two repos? Separation of concerns: the pipeline is a batch data job (Python, runs occasionally, heavy deps like pdfplumber); the frontend is a web app (Node, deployed continuously). They have different runtimes, different deploy cadences, and different dependency trees. The contract between them is just a JSON file.

The tradeoff (be honest in interview): there's *no automation* between them. Regenerating data means running the pipeline, then hand-copying two JSON files into the web repo, committing and deploying. It's a deliberate simplicity choice for a solo project — the data changes rarely — but it's the first thing I'd automate at scale (see Part 12).

PART 3 — THE DATA PIPELINE (Python)

3.1 Stack

- **Python 3**, no framework — plain scripts
- requests (HTTP), python-dotenv (API keys), pdfplumber (PDF parsing), BeautifulSoup4 (HTML parsing), tqdm (progress)
- Output: JSON files on disk. No database (see Part 12 for why, and when I'd add one)

3.2 The scraper pattern

Every dimension is a module in scrapers/ exposing a `run()` that returns a list of dicts, one per PIN, and saves both a raw and a processed copy:

```
def run():
    scraped_at = datetime.now().isoformat()
    records = [{**r, "scraped_at": scraped_at} for r in WATER_DATA]
    save_raw(records, "water") # timestamped archive
    path = save_processed(records, "water") # water_latest.json
    return records
```

Honest point that interviewers respect: only *air quality* is genuinely scraped live. The other dimensions are `STATIC_DATA` — hand-compiled constants transcribed from published government reports (Delhi Police Annual Report, DJB/BWSSB supply data, DISCOM outage reports, municipal road surveys). They're labelled "Est. 2023/2024" in the UI. This is a data-availability constraint, not a shortcut: most Indian civic data is published as PDFs and tables, not APIs.

3.3 The one real scraper: `air_quality.py`

- Hits **data.gov.in**'s CPCB real-time AQI resource (`/resource/3b01bcb8-0b14-4abf-b6f2-c1bfd384ba69`) with an API key from `.env`
- Fetches per state: Delhi, Haryana, Uttar Pradesh, Karnataka
- **Pagination:** loops offset in pages of 500 until offset `>= total`
- **Station -> PIN mapping:** `STATION_PIN_MAP` maps station-name substrings to PIN codes ("BTM Layout" -> "560076"). `resolve_pin()` does a case-insensitive substring match because the API's station names are inconsistent.
- **Fallback logic:** `FALLBACK_AQI_PINS` maps PINs with no local station to the nearest monitored PIN, so an area without a sensor still gets a real (borrowed) reading rather than nothing. Labelled [Nearest] in the record.
- **Graceful degradation:** if the fetch fails, it logs a warning, returns `[]` and the merge keeps the previous data. A network failure never corrupts the site.

3.4 The merge step (`run_pipeline.py`)

Builds one master record per PIN using a `defaultdict`:

```
master = defaultdict(lambda: {"pin_code": None, "sources": []})
```

Each source contributes fields; `sources[]` accumulates so we know provenance, and `data_completeness` = number of distinct sources for that PIN.

The bug I found and fixed here — a great interview story: `water.py`, `roads.py` and `sewerage.py` all emitted generically-named fields: `quality_score` and `coverage_pct`. The merge did a blind `dict.update()`, so whichever source ran last silently overwrote the others. That meant `score_water()` and `score_roads()` were reading *the same collided value* — two different dimensions scoring off one number.

Fix: namespace the colliding keys during the merge, keeping the originals for backward compatibility:

```
RENAME = {
    "water": {"quality_score": "water_quality", "coverage_pct": "water_coverage"},
    "roads": {"quality_score": "road_quality"},
}
```

```
"sewerage": {"coverage_pct": "sewerage_coverage"},
}
```

Verified real: for Hauz Khas, water coverage is 96% but sewerage coverage is 95% — the old code conflated them.

PART 4 — THE SCORING ENGINE (scoring.py) — *the core IP*

4.1 The composite formula

$NQI = \text{round}(\sum(\text{score}_i \times \text{weight}_i) / \sum(\text{weight}_i))$ for dimensions with data

Weights (sum to 1.00):

Dimension	Weight
Safety (crime)	25%
Infrastructure	20%
Air quality	15%
Power	10%
Schools	10%
Water	8%
Roads	7%
Drainage/Sewerage	5%

The key design decision — missing data is *dropped*, not zeroed. If a PIN has no air-quality station, air isn't scored 0 (which would unfairly tank the area); it's excluded and the remaining weights are **re-normalised to sum to 1**. Both the base and the applied weights are published per record:

```
weights_applied = {k: round(WEIGHTS[k] / total_weight, 4) for k in available}
composite = round(sum(available[k] * WEIGHTS[k] for k in available) / total_weight)
```

The UI then shows a strikethrough of the original weight next to the rescaled one, so users can see when a score leaned on fewer dimensions. This fixed a real transparency issue ("silent reweighting").

Grades: [(90, "A+"), (80, "A"), (70, "B+"), (60, "B"), (50, "C+"), (40, "C"), (0, "D")]

4.2 Per-dimension formulas (know these cold)

Crime — inverse linear normalisation with clamping:

```
LOW, HIGH = 250, 650
clamped = max(LOW, min(HIGH, crimes))
return round((1 - (clamped - LOW) / (HIGH - LOW)) * 100)
```

≤250 crimes -> 100; ≥650 -> 0; linear between. Clamping prevents outliers from distorting the scale.

Air — banded on CPCB's official AQI categories (not linear, because health impact isn't linear):

≤50 -> 100 | ≤100 -> 85 | ≤150 -> 70 | ≤200 -> 50 | ≤300 -> 30 | ≤400 -> 15 | else 5

Power — weighted blend: 60% outage frequency, 40% outage duration:

```
freq_score = (freq / 5) * 60
hour_score = max(0, (1 - (hours or 4)/8)) * 40
```

Water — 40 pts supply hours (22h = full), 35 pts quality (1-5), 25 pts coverage
Roads — 60 pts surface quality (1-5), 40 pts inverse pothole density (capped at 20/km)
Sewerage — 40 pts coverage, 35 pts waterlogging risk, 25 pts treatment adequacy
Schools — up to 60 pts density + 40 pts share CBSE/ICSE-recognised
Infrastructure — passthrough of a precomputed composite (metro proximity, highway access, zone type, smart-city status)

4.3 Crime percentile (context, not just a raw number)

A raw “412 crimes” is meaningless without comparison. So each PIN gets a percentile rank and a tier (Very Low -> Very High).

Critically, it’s city-scoped:

`counts_by_city = defaultdict(list)` *# rank within Delhi, or within Bangalore*

Because Delhi Police and Bengaluru City Police publish on different scales, ranking a Bangalore area against Delhi counts would be apples-to-oranges.

Tie handling: only PINs with a *strictly higher* count count as “less safe”, so tied PINs get no credit either way.

4.4 What I deliberately *didn’t* build — the false-precision principle

This is the single best thing to talk about in an interview because it shows judgment over cleverness.

1. **Per-capita crime rate.** The obvious “improvement” is crimes per 1,000 residents. I researched it and rejected it: Census 2011 only publishes population down to *tehsil* level in Delhi — ~27 tehsils covering ~55 tracked PINs. Multiple different neighbourhoods would share an identical denominator. That’s not more accurate, it’s the same ranking dressed up in units that *look* rigorous. Percentile ranking is the honest ceiling.
2. **Exact per-PIN property prices.** Karnataka’s guidance value is set per *street/survey number*, not per PIN, and a PIN spans several valuation zones. So an “exact price per PIN” doesn’t exist. I ship a labelled *indicative band* instead.
3. **Trend/trajectory data.** Requires time-series; the underlying data is static snapshots. Can’t fake it.

The principle: never let the presentation imply more precision than the data supports.

4.5 Published methodology

`save_methodology()` writes `methodology.json` — every dimension’s weight, formula (as a string) and a plain-English description — so the score is externally re-derivable. The UI renders it. This directly addressed “no visible methodology” feedback.

PART 5 — PRICE CONTEXT (a non-composite dimension)

Brokers and investors said the livability score ignored price. So I added price context — but **deliberately NOT in the NQI**, exactly like crime percentile: it’s informational, because “expensive” isn’t “good” or “bad”.

Data source: government minimum valuation rates — - **Delhi:** MCD **circle rates**, categories A–H (A = ₹7.74 lakh/sqm land, H = ₹23,280/sqm) - **Haryana** (Gurugram/Faridabad): **collector rates** — same concept, different name - **UP** (Noida/Ghaziabad): circle rates, sector categories - **Karnataka** (Bangalore): **guidance value** via the Kaveri portal

Buyer-facing design: - Everything normalised to **₹/sq ft** (buyers don’t think in ₹/sqm) - Ordinal band: Premium -> Upper -> Mid -> Modest -> Value - Explicit “circle rate = collector rate = guidance value, they’re the same thing” - A **market-gap explainer:** market prices run 20–70% above the government rate, with a worked example (“a ~1,000 sq ft flat here ≈ ₹1.4–2.2 cr”) and the note that home loans cap near the circle-rate value, so the gap is your own funds.

A mistake I caught and corrected — good honesty story: my first Bangalore figures were too high because I’d sourced them from real-estate articles that blur “guidance value” and “market rate”. Whitefield showed ₹9,000–14,000 when the actual guidance value is ~₹6,500. Worse, the UI *then* added “market is 20–70% higher” on top — double-counting the inflation. I recalibrated the whole set against verified anchors (Indiranagar ~₹8k, Whitefield ~₹6.5k, EC ~₹6k).

PART 6 — THE FRONTEND (Next.js)

6.1 Stack

- **Next.js 16.2.1** (Pages Router), **React 19.2.4**
- **recharts** — radar chart for the dimension overview
- **gsap, three** — animation/3D on the landing page
- **@vercel/og** — dynamic Open Graph images
- **nodemailer** — feedback email delivery
- **Leaflet** (CDN) — the interactive map
- **jsPDF + html2canvas** (CDN) — PDF export
- ESLint (eslint-config-next), TypeScript available but pages are .js

Why Pages Router not App Router? The project started on Pages Router and it does everything needed here — `getServerSideProps` gives per-request SSR for SEO, which is the main requirement. Migrating for its own sake would be churn.

6.2 Rendering strategy — SSR, and why it matters

`pages/report/[pin].js` uses `getServerSideProps`:

```
export async function getServerSideProps({ params, req }) {
  const pin = params?.pin
  if (!pin || !/^\d{6}$/.test(pin)) return { redirect: {...} } // validate
  const [r1, r2] = await Promise.all([                          // parallel
    fetch(`${base}/api/report?pin=${pin}`),
    fetch(`${base}/api/all`),
  ])
  return { props: { report, allScores, ogMeta } }
}
```

Why SSR rather than client fetch? SEO. These are content pages that must be crawlable — the score, grade and area name have to be in the initial HTML, along with OG tags for link previews and JSON-LD structured data. A client-side fetch would serve an empty shell to crawlers.

Note `Promise.all` — the two fetches are independent, so they run in parallel, halving latency.

6.3 API routes

- `/api/report?pin=` — finds the PIN in `nqi_scores.json`, merges the matching `master_by_pin.json` record, returns `{...score, ...master}`. This merge is why the report page has both computed scores *and* raw stats.
- `/api/all` — the full scores array (used for nearby-area comparison)
- `/api/og` — dynamic OG image generation via `@vercel/og`
- `/api/feedback` — POSTs feedback to email via `nodemailer` + Gmail SMTP (`EMAIL_USER` / `EMAIL_PASS` as Vercel env vars). Silent submission — no `mailto:` popup.

6.4 The report page — key features

Interactive re-weighting (the standout feature). Users pick a persona — Default / Family / Investor / Safety / Custom — and the NQI recomputes live:

```
function normalizedWeights(weights, availableKeys) {
  const total = availableKeys.reduce((s,k) => s + (weights[k]||0), 0)
  return Object.fromEntries(availableKeys.map(k => [k, (weights[k]||0)/total]))
}
```

Custom mode gives 8 sliders. Weights are normalised at compute time rather than forcing sliders to interlock, so dragging one never fights the others. The “official” score is always shown alongside “your score” so the canonical number isn’t lost.

Other features: shortlist (localStorage-persisted, with a “compare top 2” link that pre-fills the compare page via ?a=&b=), PDF export, WhatsApp/native share, plain-language AQI, data-freshness strip, waterlogging risk surfacing, feedback form, dark/light mode, city switcher.

6.5 Performance & correctness details worth mentioning

- useMemo for the NQI recomputation so slider drags don’t recompute everything
 - Leaflet loaded **lazily from CDN** on demand, not bundled — keeps the bundle small
 - The map effect has a cleanup (map.remove()) to prevent leaks on re-render, and re-initialises when the theme or unlock state changes
 - localStorage reads are wrapped in try/catch (Safari private mode throws)
-

PART 7 — THE “INDUSTRY” REDESIGN

A full visual redesign of the report page from a design handoff — a blueprint/spec-sheet aesthetic: hairline-framed cards with + corner registration marks, square corners (radius 0), no shadows, Barlow Condensed headings, single maroon accent #7a1f2b.

Design-token architecture — every colour derives from one accent variable plus a mode flag, using CSS color-mix():

```
.iv {
  --acc-base:#7a1f2b;
  --acc: color-mix(in srgb, var(--acc-base) 55%, #dfa3ab); /* readable on dark */
  --acc-deep: color-mix(in srgb, var(--acc-base) 35%, #f0c9cd);
  --acc-fill: var(--acc-base); /* solid fills */
}
.iv.light { --bg:#f2f2f3; --ink:#1d1f20; --acc:var(--acc-base); }
```

Components never hard-code mode-specific colours — dark/light is one class swap.

Risk management (important to articulate): the live page was ~2,000 lines with every feature interwoven. Rewriting it in place would have left production broken mid-way. So I built the redesign at a **separate route** (/report-v2/[pin]), verified it whole, then swapped /report over — preserving the original page’s SEO/OG/JSON-LD — and kept the old file as legacy/report-pin-legacy.js.bak for instant rollback.

Full-UI PDF export: rather than a text summary, html2canvas captures the actual rendered page, then it’s sliced into multi-page A4 via jsPDF. It auto-unlocks the gated section first, hides interactive-only elements ([data-pdf="1"] .no-pdf { display:none }) and matches the current theme.

PART 8 — DEPLOYMENT & OPS

- **Hosting:** Vercel. **Critical quirk:** the project was deployed via Vercel CLI, *not* GitHub integration — so git push does **not** deploy. You must run npx vercel --prod. (Interview point: I’d fix this by connecting the Git integration for proper CI/CD with preview deployments per PR.)
 - **Edge caching** is aggressive; verify with a ?cb= cache-buster.
 - **DNS incident:** the site once went blank globally — deployments were all healthy, but the Vercel dashboard showed the domain as “Invalid Configuration”. The A record (216.198.79.1) / CNAME (...vercel-dns-017.com) had broken at the registrar. Diagnosis path: confirm it’s not local (test on cellular) -> check whether it’s the *build* or the *domain* -> fix DNS. Good “how do you debug production” story.
 - **Licence:** proprietary “all rights reserved” (BlindSpot), in both repos.
-

PART 9 — THE MULTI-CITY EXPANSION (Delhi -> Bangalore)

The scoring engine was already city-agnostic, so the work was data + UX.

What I had to build: 1. A Bengaluru seed module (scrapers/bengaluru_data.py) — 66 real localities with a tier-based baseline + per-area overrides, injected at the merge step 2. city tagging on every record (560xxx -> Bangalore) flowing into the scores 3. City-scoped crime percentile (Part 4.3) 4. Frontend: Bengaluru PIN metadata in all four PIN_META copies, a city switcher, city-aware source labels (BESCOM vs BSES, BWSSB vs DJB, BBMP vs MCD), and city-aware price wording (guidance value vs circle rate)

What came free: air quality (the CPCB API is national — just add Karnataka + station mappings) and schools (the CBSE affiliation CSV is national — it already had 126 Bangalore PINs). So only 5 of 8 dimensions needed hand-compilation.

A validation story: after building it, I programmatically checked all 67 areas against authoritative India Post data instead of eyeballing. Found 8 real errors: 560016/560036 were swapped (KR Puram ↔ Ramamurthy Nagar), 560052 was mislabelled, 560051/560052 collided, 560024 was wrong, 560046 was wrong, and 560082 wasn't an urban area at all (rural villages off Kanakapura Road) so I dropped it — 67 -> 66.

PART 10 — LIKELY INTERVIEW QUESTIONS & STRONG ANSWERS

Q: Why is this “just JSON files” and not a database? Scale and access pattern. 152 records, read-only, changing maybe monthly. A DB would add infrastructure, latency and cost for zero benefit — the whole dataset is smaller than a single product image. Vercel serves the JSON from its edge. I'd move to Postgres the moment I need per-user data (saved shortlists across devices), historical time-series for trends, or writes.

Q: How do you handle missing data? Drop-and-renormalise, never zero-fill. Zeroing would punish an area for a sensor we don't have. I also expose `dimensions_scored / dimensions_total` and both base and applied weights so the user sees exactly how complete their score is.

Q: How do you know your scores are right? I don't claim they're “right” — I claim they're *transparent and reproducible*. The methodology is published as JSON with every formula, so anyone can re-derive the number. Where data can't support a claim, I say so in the UI rather than inventing precision.

Q: What's the weakest part of this system? Data freshness. Six of eight dimensions are hand-transcribed static snapshots dated 2023-24. They're clearly labelled “Est.”, but the honest answer is the product's ceiling is the availability of Indian civic data as machine-readable feeds. The fix isn't cleverer code — it's periodic manual refresh, or partnering for data access.

Q: Why aren't the weights validated against anything? They're expert-assigned priors, not learned. There's no ground-truth “livability” label to regress against. That's exactly why I built user re-weighting — instead of pretending one weighting is objective, I let the user impose their own and show both numbers.

Q: Biggest bug you found? The field-collision bug in the merge (Part 3.4) — two dimensions silently scoring off the same value because of a blind `dict.update()` with generic key names. Lesson: namespace fields at source boundaries; generic names like `quality_score` are a landmine in a multi-source merge.

Q: How would you scale this to 20 cities? The engine already generalises. The bottleneck is per-city data compilation, so I'd: (1) build a city config abstraction (agencies, price system, station map) so adding a city is data not code; (2) automate the pipeline->web handoff via CI; (3) move to a database with a proper schema and migrations; (4) add automated validation — the PIN-to-locality check I ran manually should be a test.

Q: Why not use an LLM to generate the missing data? Because it would be fabrication. The entire value proposition is “real government data, honestly labelled”. Plausible-sounding invented numbers would be worse than no product.

PART 11 — GLOSSARY (don’t get caught out)

- **NQI** — Neighbourhood Quality Index, the 0–100 composite
 - **PIN code** — 6-digit Indian postal code; the unit of analysis
 - **CPCB** — Central Pollution Control Board (national air quality)
 - **AQI** — Air Quality Index; CPCB bands: Good ≤ 50 -> Severe > 400
 - **Cognizable crime** — offences where police can arrest without a warrant
 - **Circle rate / collector rate / guidance value** — the same concept under three state names: the government’s *minimum* property valuation for stamp duty
 - **TDS** — Total Dissolved Solids, a water-quality measure
 - **DISCOM** — electricity distribution company (BSES/Tata Power in Delhi, BESCOM in Bangalore)
 - **DJB / BWSSB** — Delhi Jal Board / Bangalore Water Supply & Sewerage Board
 - **BBMP / MCD** — municipal corporations of Bangalore / Delhi
 - **Waterlogging risk** — 1–5 where **5 = safest** (inverted; a real footgun in the code)
-

PART 12 — WHAT I’D DO NEXT (shows product thinking)

1. **Automate the pipeline->web handoff** — a GitHub Action that runs the pipeline on a schedule, commits the JSON, and triggers a deploy. Removes the manual step.
 2. **Connect Vercel’s Git integration** for real CI/CD and preview deploys.
 3. **Add tests** — the PIN-to-locality validation and the scoring formulas should be a pytest suite, not a one-off script.
 4. **Database + user accounts** — persistent shortlists across devices, saved searches, alerts when an area’s score changes.
 5. **Sub-PIN granularity** — the biggest known limitation: one PIN can span a quiet colony and a commercial strip, scored identically. Needs ward/locality polygons.
 6. **Time-series** — start snapshotting now so trend analysis becomes possible later.
 7. **Live air for Bangalore** — code is written; blocked only on API reachability.
-

PART 13 — THE 3 THINGS TO LEAD WITH IN AN INTERVIEW

1. **The false-precision principle.** I twice researched an “obvious” feature (per-capita crime, exact per-PIN prices), found the data couldn’t support it, and *chose not to ship it*. Engineering judgment is knowing what not to build.
2. **The silent-collision bug.** Two dimensions scoring off the same value due to a blind merge with generic key names — found it by auditing, fixed it by namespacing at the source boundary, verified the difference was real.
3. **Shipping a redesign without breaking production.** Built at a parallel route, verified whole, swapped atomically, preserved SEO, kept a rollback file. The live site never broke.